XC330 Simulator: Proposed Secondary-Goal Torque-Assist Architecture

Implementation-derived reference

This note describes the proposed control architecture for `OperatingMode.CURRENT_POSITION` and `Objective.SECONDARY_GOAL_ASSIST` are selected. It is a simplified model, not a numerical reproduction of XC330 firmware. The simulator now exposes this architecture as its default secondary-goal objective; the older dynamic-current cap objective remains available for comparison.

What controls what

Signal or parameter	Responsibility
$\hat{\tau}_{ext}$, bias	Motor-side estimate and zeroing; determines the debiased effort r.
v_{goal}	A filtered, signed intent velocity derived from sustained debiased effort. It controls activation and release.
q_{goal}	Persistent secondary goal position, integrated from v_{goal}. It is not recomputed as $q +$ lead each frame.
ℓ and q_{ref}	Bounded lead ahead of q_{goal}; $q_{ref} = q_{goal} + \mathrm{sgn}(v_{goal})\ell$.
τ_{PID}	Desired compliant position-control effort before current limiting; use low P, little/no I, and D for damping.
$I_{ceiling}$, τ_{act}	Independent current/safety ceiling and actual applied torque after clamping the PID request.

Discrete-time algorithm

At each simulation step of duration Δt, the proposed controller executes the following sequence. The external profile is the simulated plant disturbance; the observer does not directly read the known disturbance.

1. Generate external effort:
$$\tau_{ext,k} = \mu(t_k) + \epsilon_k + \tau_{ext,offset}.$$

2. Estimate external effort from measured acceleration, modeled friction, actuator torque, and endstop torque. Low-pass filter the residual to obtain $\hat{\tau}_{ext,k}$. During the collection epoch, estimate the bias b_k as the running mean. Then compute
$$r_k = \hat{\tau}_{ext,k} - b_k.$$

3. Form a signed intent velocity from the filtered residual:
$$v^*_{goal,k} = k_r r_k.$$
Apply a threshold and dwell timer so a short noise excursion does not activate assistance. Release when the residual is small or when progress in the intended direction remains below a velocity threshold for a release dwell.

4. Integrate a persistent secondary goal:

$$q_{goal,k+1} = q_{goal,k} + \Delta t\, v_{goal,k}.$$

   On release, freeze or capture q_{goal} at the achieved position and decay the lead state. This prevents the reference from continuing to run away when the user or a physical endstop can no longer produce motion.

5. Form a bounded target lead and the position reference:

$$q_{ref,k} = q_{goal,k} + \operatorname{sgn}(v_{goal,k})\ell_k, \qquad 0 \le \ell_k \le \ell_{max}.$$

   The soft-endstop position is not used as a clamp on q_{ref}. Measured velocity is used as a progress signal; if it collapses, lead and assistance are released instead.

6. Compute compliant position-control effort:

$$e_k = q_{ref,k} - q_k, \qquad \tau_{PID,k} = K_p e_k + K_i \int e\, dt + K_d(v_{ref,k} - v_k).$$

7. Apply an independent current/safety ceiling:

$$\tau_{act,k} = \operatorname{clamp}\left(\tau_{PID,k}, -I_{ceiling}K_t, +I_{ceiling}K_t\right).$$

   Here the assist signal changes the target and its lead, while $I_{ceiling}$ remains an independent compliance/safety parameter.

8. Advance the plant:

$$\dot{v}_k = \frac{\tau_{act,k} + \tau_{ext,k} + \tau_{endstop,k} - \tau_{friction,k}}{J}, \qquad v_{k+1} = v_k + \Delta t \dot{v}_k, \qquad q_{k+1} = q_k + \Delta t v_{k+1}.$$

Interpreting the proposed output stage

If the view shows $\tau_{PID} = -1.76$ N m and $I_{ceiling} = 300$ mA, then the maximum available torque is

$$|\tau_{act}| \le 300\,\text{mA} \times K_t \approx 0.345\,\text{N m}.$$

The applied torque is therefore the PID request clipped to about -0.345 N m, not necessarily -1.76 N m. The sign is carried by applied current/torque; the configured Goal Current is a magnitude limit.

Important design choice

The proposed design uses the assist estimate to create a secondary goal velocity and position lead. The position controller remains compliant through low P, minimal I, damping D, and an independent current ceiling. If progress stops, assistance releases and the achieved position is captured; if progress continues, the secondary goal can advance toward the configured endpoint and remain there.

Firmware reconciliation (assist-v3)

The OpenRB implementation in `assist_controller.h/.cpp` now implements this secondary-goal architecture. Defaults are $K_P^{raw} = 350$, $K_I^{raw} = 0$, $K_D^{raw} = 160$, $I_{ceiling} = 300$ mA, $\ell_{max} = 8$ degrees, lead slew 24 degrees/s, detection and stall dwell 0.5 s, progress threshold 1 degree/s and intent conversion 80 degrees/s per N m. The effort-intent ceiling is 0.25 N m; it is independent of the current ceiling.

The GUI's explicit Enable operation performs grouped selected-ID mode/hold setup, collects 50 stationary baseline samples at nominal 50 Hz (one second), and activates automatically. No manual per-joint bias workflow is required. With requested cap C, configured motor limits L_i and total current budget B, installed caps are $\lfloor \min(C, L_i) \min(1, B/\sum_j \min(C, L_j)) \rfloor$. The GUI reports these effective caps; neither safety limits nor the budget are raised. Insufficient threshold/bias/noise headroom rejects startup. Automatic activation preserves the host heartbeat deadline rather than renewing it.

All selected integer motor IDs are serviced together on each 20-ms control pass: one Protocol 2.0 Sync Read of current/velocity/position, one Sync Read of torque status, then at most one Sync Write containing every changed goal. There is no round-robin selection or per-ID fallback. Partial replies or status errors reject the whole frame before any goal write. Motors reply serially on the bus; a shared frame timestamp does not imply simultaneous physical sampling. Calibration settings are installed and read back in groups before torque-on; stopping uses independent zero-current and torque-off Sync Writes followed by a grouped torque-status read to confirm actual disable. The firmware observer filters

$$r_I^* = (J_d a_d + B_d v_d + C \operatorname{sgn}(v_d))/K_t - I + b_I, \qquad \alpha = \frac{\Delta t}{0.20 + \Delta t}.$$

The configurable hardware model terms default to zero, giving a quasi-static holding-current residual proxy. Simulator plant parameters are illustrative, not identified motor/linkage dynamics. The simulator's inertia coefficient multiplies acceleration in degrees/s^2; a physical SI inertia requires a $\pi/180$ conversion for this degree-coordinate equation. No environmental endstop torque is independently measured or subtracted on hardware. Environmental resistance is part of its residual load; stalled progress triggers release. There is no firmware soft-endstop addition.

Stored hard joint limits plus a 2-degree margin bound both the integrated goal and emitted reference, including inward encoder quantization. Multi-turn wrist coordinates are preserved. Firmware captures the achieved pose on effort release as well as stalled progress, and cancels lead immediately on the release observation. These are intentional refinements: the simulator can retain lead for one stall-transition sample and decay it against an old goal on ordinary release. After capture, 0.5 s below half the effort threshold is required to rearm. The current ceiling remains fixed throughout the session.

Bias collection, per-ID exclusivity, measured-feedback faults, heartbeat lease, and torque-off retry remain device-side guards. The previous 5-degree bias window and discrete step/settle gates are removed; a fixed bias still does not identify gravity throughout ROM. Matching raw PID registers does not make the simplified simulation a numerical model of the motor's internal controller. See `docs/assist_mode.md` for SDK units, protocol and verification.